```
        format.html { render :new, status: :unprocessable_entity }
        format.json { render json: @order.errors,
          status: :unprocessable_entity }
      end
    end
  end
```



Joe asks:

Aren't You Creating Duplicate Orders?

Joe is concerned to see our controller creating `Order` model objects in two actions: `new` and `create`. He's wondering why this doesn't lead to duplicate orders in the database.

The answer is that the `new` action creates an `Order` object *in memory* simply to give the template code something to work with. Once the response is sent to the browser, that particular object gets abandoned, and it'll eventually be reaped by Ruby's garbage collector. It never gets close to the database.

The `create` action also creates an `Order` object, populating it from the form fields. This object *does* get saved in the database. So model objects perform two roles: they map data into and out of the database, but they're also regular objects that hold business data. They affect the database only when you tell them to, typically by calling `save`.

We start by creating a new `Order` object and initialize it from the form data. The next line adds into this order the items that are already stored in the cart; we'll write the method to do that in a minute.

Next, we tell the order object to save itself (and its children, the line items) to the database. Along the way, the order object will perform validation (but we'll get to that in a minute).

If the save succeeds, we do two things. First, we ready ourselves for this customer's next order by deleting the cart from the session. Then we redisplay the catalog, using the `redirect_to` method to display a cheerful message. If, instead, the save fails, we redisplay the checkout form with the current cart.